Servlet + JSP Build Promotion Demo

Complete End-to-End Explanation & Flow Guide

Technology: Java 21 • Maven • Embedded Tomcat • Jakarta Servlet • JSP • JDBC • MySQL

Prepared as a manager-demo / interview revision handbook

1. What We Built

We built a small Build Promotion web application using Java Servlets and JSP. The application accepts an application name, build version and target environment, processes and prefixes the values, stores them in MySQL, maintains session information, creates a cookie, and displays the result in JSP.

Browser
↓
index.jsp — input form
↓ HTTP GET / POST
Embedded Tomcat
↓
BuildPromotionServlet
↓
MySQL + Session + Cookie
↓
result.jsp — output UI

Project Files

File	Purpose
pom.xml	Maven project configuration and dependencies
Main.java	Starts and configures embedded Tomcat
BuildPromotionServlet.java	Servlet lifecycle, GET/POST, DB, session and cookie handling
index.jsp	Input page and GET/POST entry point
result.jsp	Displays processed values, session, cookie and DB records

2. Maven and pom.xml

Maven manages the project dependencies and build configuration. We use Java 21 and only the libraries required for this simple Servlet/JSP application.

Dependency	Why it is needed
tomcat-embed-core	Runs embedded Tomcat and provides Servlet infrastructure.
tomcat-embed-jasper	Processes and compiles JSP pages.
mysql-connector-j	JDBC driver that lets Java communicate with MySQL.

Manager Questions

Why embedded Tomcat?

> We want Tomcat to run inside our Java application instead of requiring a separately installed and manually started Tomcat server.

Why Jasper?

> Jasper is Tomcat's JSP engine. We need it because the application contains index.jsp and result.jsp.

Why MySQL Connector/J?

> It is the MySQL JDBC driver used by DriverManager and JDBC to establish the database connection.

3. Main.java — Starting Embedded Tomcat

Main.java is the application entry point. It creates Tomcat, configures the port and web application, registers the Servlet mapping, starts Tomcat, and keeps the server alive.

Main.main()
↓
Create Tomcat instance
↓
Set port 8080 + initialize connector
↓
Mount src/main/webapp as /ServletDemo
↓
Register BuildPromotionServlet
↓
Map /promote → BuildPromotionServlet
↓
tomcat.start()
↓
server.await()

URL

```
http://localhost:8080/ServletDemo/promote
```

What is the context path?

/ServletDemo is the web application's context path. **/promote** is the Servlet URL mapping.

4. Servlet Registration and Routing

The Servlet is declared with @WebServlet("/promote") and is also explicitly registered in Main.java for the embedded Tomcat setup. The explicit registration tells the embedded container exactly which Servlet object should handle the /promote mapping.

HTTP request: /ServletDemo/promote
↓
Tomcat receives request
↓
Application context: /ServletDemo
↓
Servlet mapping: /promote
↓
BuildPromotionServlet

5. Servlet Lifecycle

The Servlet container manages the lifecycle. For this demo, the important stages are initialization, request processing, and destruction.

Servlet class available
↓
Servlet object created
↓
init() — once for the Servlet instance
↓
Servlet ready
↓
doGet() / doPost() — for incoming requests
↓
Application/server shutdown or Servlet removal
↓
destroy() — cleanup

Why init()?

Our manager specifically asked for the DB connection during init(). We establish the JDBC connection and create/verify the database table when the Servlet is initialized.

Why destroy()?

destroy() is used for cleanup. Our implementation closes the JDBC connection when the Servlet is taken out of service.

6. Database Initialization in init()

We do not need to manually create the database or table in MySQL Workbench. init() first connects to the MySQL server, creates the database if needed, then connects to the application database and creates the table if needed.

init()
↓
Connect to MySQL server
↓
CREATE DATABASE IF NOT EXISTS servlet_demo
↓
Connect to servlet_demo
↓
CREATE TABLE IF NOT EXISTS build_promotion
↓
Servlet ready

Database Structure

```
MySQL
■■■ servlet_demo
■■■ build_promotion
■■■ id
■■■ application
■■■ version
■■■ environment
■■■ created_at
```

Database: servlet_demo **Table:** build_promotion

7. JSP Input Page — index.jsp

index.jsp is the UI where the user enters the build promotion data.

```
<form action="promote" method="POST">

Application: <input name="application">
Build Version: <input name="version">
Environment: <select name="environment">...</select>

<button type="submit">Promote Build</button>
</form>
```

The HTML field name must match the parameter name used in the Servlet. For example, name="version" is read using request.getParameter("version").

8. GET Request Flow

GET is used in our application to retrieve and display existing promotion records.

Browser opens /ServletDemo/promote
↓
GET request
↓
Tomcat → BuildPromotionServlet.doGet()
↓
Query existing records from build_promotion
↓
Put records into request attribute
↓
Forward to index.jsp
↓
Existing promotions displayed

Manager answer: GET is used for retrieving/displaying existing data. It does not create a new promotion record.

9. POST Request Flow

POST is used when the user submits a new build promotion.

User fills Application + Version + Environment
↓
Submit form
↓
POST /ServletDemo/promote
↓
doPost()
↓
request.getParameter(...)
↓
Prefix values

↓
INSERT into build_promotion
↓
Create/update session
↓
Set cookie
↓
Fetch DB records
↓
forward() → result.jsp

10. Prefixing the Input

The manager asked us to prefix incoming values before showing and storing them.

Input	Processed value
Payment Service	APP-PAYMENT SERVICE
2.4.1	BUILD-2.4.1
QA	TARGET-QA

The Servlet performs the transformation using Java strings and then uses the processed values for the DB insert and UI.

11. JDBC Insert and PreparedStatement

```
String insertSQL =
"INSERT INTO build_promotion " +
"(application, version, environment) " +
"VALUES (?, ?, ?)";

PreparedStatement ps =
dbConnection.prepareStatement(insertSQL);

ps.setString(1, prefixedApplication);
ps.setString(2, prefixedVersion);
ps.setString(3, prefixedEnvironment);

ps.executeUpdate();
```

Why PreparedStatement?

> It supports parameterized SQL, separates values from the SQL statement, makes the code cleaner, and helps protect against SQL injection.

12. Session

HTTP is stateless, so a session provides a way to associate data with a user across multiple requests.

```
HttpSession session = request.getSession(true);

session.setAttribute(
"sessionApplication",
prefixedApplication
);

session.setAttribute(
"sessionEnvironment",
prefixedEnvironment
);
```

First request
↓
getSession(true)
↓
Session created or existing session returned
↓
Session attributes stored
↓
Later request
↓
Same session can be used to retrieve those attributes

13. Cookie

A cookie is small data stored by the browser and sent back with later requests.

```
Cookie applicationCookie =
new Cookie("lastApplication", cookieValue);

applicationCookie.setMaxAge(86400);

response.addCookie(applicationCookie);
```

Servlet
↓ response.addCookie()
HTTP Response
↓
Browser stores cookie
↓
Next HTTP request
↓

Browser sends Cookie header
↓
Servlet can read request.getCookies()

Important: A cookie created in the current response normally becomes available to the server on a subsequent request, because the browser must first store it and then send it back.

14. Session vs Cookie

Session	Cookie
Primarily maintained on the server	Stored by the browser/client
Can hold server-side user state	Small key/value data sent with requests
Identified through a session mechanism/ID	Sent through HTTP Cookie headers
Used in our app for application/environment session data	Used in our app for lastApplication

15. Passing Data from Servlet to JSP

After processing, the Servlet stores values in the current request and forwards to result.jsp.

```
request.setAttribute(
"prefixedApplication",
prefixedApplication
);

request.setAttribute(
"promotionList",
promotionList
);

request.getRequestDispatcher(
"/result.jsp"
).forward(request, response);
```

BuildPromotionServlet
↓
request.setAttribute(...)
↓
RequestDispatcher.forward(...)
↓
result.jsp
↓
HTML response to browser

16. result.jsp

result.jsp is the output view. It displays:

Section	What is shown
Processed values	Prefixed application, version and environment
Session	Session ID, application and environment
Cookie	Current cookie information / received cookies
Database	Promotion records fetched from MySQL

17. Complete End-to-End Flow

1. User opens application
↓
2. Embedded Tomcat receives HTTP request
↓
3. URL /ServletDemo/promote maps to BuildPromotionServlet
↓
4. Servlet lifecycle: init() initializes DB
↓
5. GET → retrieve/display existing records
OR
POST → process a new promotion
↓
6. Read request parameters
↓
7. Prefix values
↓
8. Save to MySQL using PreparedStatement
↓
9. Maintain HttpSession
↓
10. Add Cookie to response
↓
11. Fetch promotion records

↓
12. request.setAttribute(...)
↓
13. forward() to result.jsp
↓
14. JSP renders HTML
↓
15. Browser displays result
↓
16. On shutdown/removal → destroy() → close DB connection

18. Lifecycle vs Request Flow

These two flows are easy to confuse. Keep them separate.

Servlet Lifecycle	HTTP Request Flow
init()	doGet() / doPost()
Called when Servlet is initialized	Called when a request arrives
Used for initialization	Used for request processing
Our DB setup happens here	Our GET/POST logic happens here
destroy()	Ends when request processing finishes

19. Manager Rapid-Fire Questions

What is a Servlet?

A Java server-side component that processes HTTP requests and generates HTTP responses. The Servlet container manages its lifecycle.

Who manages the Servlet lifecycle?

The Servlet container. In our project, that is embedded Tomcat.

What are the main lifecycle stages?

init(), request processing such as doGet()/doPost(), and destroy().

When is init() called?

When the Servlet is initialized by the container, generally once for that Servlet instance.

When is destroy() called?

When the Servlet is taken out of service, commonly during application/server shutdown or reload.

Why embedded Tomcat?

It lets us run Tomcat inside our Java application without a separately installed Tomcat server.

What is /ServletDemo?

The application context path.

What is /promote?

The URL mapping for BuildPromotionServlet.

What does GET do?

Retrieves and displays existing promotion records.

What does POST do?

Receives a new promotion, processes it, stores it in MySQL, manages session/cookie data, and forwards to the result page.

Why POST for submission?

The request is sending data to the server for processing and database creation. POST is appropriate for this operation.

Why JDBC?

JDBC is Java's standard API for interacting with relational databases.

Why MySQL Connector/J?

It is the MySQL-specific JDBC driver used by Java/JDBC.

Why PreparedStatement?

It supports parameterized SQL and helps prevent SQL injection.

Why Session?

To maintain user-specific state across multiple HTTP requests.

What is a Cookie?

Small data stored by the browser and sent back with subsequent requests.

Session vs Cookie?

Session state is primarily maintained server-side; cookies are stored client-side in the browser.

Why JSP?

It is the view technology used to generate/display dynamic HTML.

Why Jasper?

Jasper is Tomcat's JSP engine used to process JSP pages.

Why request.setAttribute()?

To pass data from the Servlet to the JSP through the current request.

Why forward()?

> To transfer processing to the JSP while keeping the same request and response objects.

What is the DB name?

> servlet_demo.

What is the table name?

> build_promotion.

20. One-Minute Project Explanation

Use this when the manager says: **"Explain your project."**

I created a simple Servlet and JSP based Build Promotion application using Java 21, embedded Tomcat, JDBC and MySQL. The application has a JSP form where the user enters the application, build version and environment. Tomcat routes the request to BuildPromotionServlet using the /promote mapping. During Servlet initialization, init() establishes the MySQL connection and creates the database and table if they do not exist. For GET requests, we retrieve and display existing promotion records. For POST requests, we receive the submitted values, prefix them, store them in MySQL using PreparedStatement, create session attributes and a cookie, and then forward the processed data to result.jsp. Finally, when the Servlet is taken out of service, destroy() closes the database connection.

21. Demo Checklist

Check	Expected
Maven dependencies downloaded	Build succeeds
MySQL running	JDBC connection succeeds
Password updated	init() connects successfully
Database	servlet_demo created automatically
Table	build_promotion created automatically
Tomcat	Starts on port 8080
GET	Existing records displayed
POST	New record inserted
Prefixing	APP-, BUILD-, TARGET- visible
Session	Session values visible
Cookie	Cookie visible on subsequent request
destroy()	DB connection closes on shutdown

22. Final Mental Model

Maven
↓ provides required libraries
Embedded Tomcat

↓ manages Servlet lifecycle + HTTP
BuildPromotionServlet
↓ init() → DB setup
↓ doGet() → read/display existing data
↓ doPost() → process new data
↓ Session + Cookie
↓ JDBC → MySQL
↓ forward()
JSP
↓
HTML
↓
Browser

If you remember this chain, you can explain the project without memorizing every line of code.